



Agentic AI Learners' Space 2026

Assignment 3

Topics: MCP, LangChain and LangGraph

Submission Deadline: 30th June 11:59 pm

Hello! We hope you enjoyed the Week 3 resources on Model Context Protocol (MCP), LangChain and LangGraph. This assignment will test your conceptual understanding and practical skills from those materials. Follow through the provided instructions. Good luck!

Instructions:

- The Assignment contains 3 Questions. You can write your Python code in either Jupyter notebook, Pycharm, VS Code, or any other code editor of your choice.
- For submission, either submit the **pdf** of a single .ipynb (Jupyter Notebook) or submit the pdf file of a word document in which you will copy/paste your python code along with the screenshot of the respective output. Name the final submission pdf file as: AgenticAI_Week3_<YourName>_<RollNo>
- All three questions must be attempted. Code must run end-to-end without errors.

Question 1: Build a ReAct Agent with LangChain

Build a ReAct agent using LangChain that can answer questions requiring tool use.

Your agent must have **at least 2 tools** of your choice. For example: a calculator, a web search tool, a weather lookup, a currency converter, a Wikipedia fetcher, etc. (use DuckDuckGoSearchRun from langchain_community. It is a free drop-in for web search with no API key needed).

The agent should:

1. Accept a user query as a string input.

2. Reason through which tool(s) to call using the ReAct pattern (Thought → Action → Observation).
3. Return a final answer after one or more tool calls.

Demonstrate your agent working on **3 different queries** that together force it to use both tools at least once each. Print the full verbose output.

Question 2: Build a LangGraph Workflow with Cycles

Build a LangGraph agent that answers a research question through an iterative loop. It must loop at least once before terminating.

The workflow must have **at least 3 nodes** and use a **conditional edge** to decide whether to loop or stop. A suggested (but not required) structure:

- generate node: LLM generates an answer or partial answer.
- evaluate node: LLM (or a simple heuristic) checks whether the answer is good enough (e.g. is it longer than N words? Does it contain certain keywords? Does it cite a source?)
- refine node: LLM improves the answer if evaluation fails.

Conditional edge: after evaluate, if the answer passes → END; if not → back to refine (loop).

Define your AgentState as a TypedDict. Print the final state at the end.

You may use any free LLM for this question. **Note:** You can also use a local model via Ollama.

Question 3: Cold Email Agent via Gmail MCP

Build a LangGraph agent that drafts and sends cold emails to professors or companies using the **Gmail MCP server**.

MCP setup:

Use langchain_mcp_adapters to connect to Gmail's MCP server. A free Gmail MCP server is available at:

`npx @gptscript-ai/gmail-mcp-server`

Alternatively, you can use the stdio-based MCP approach with the Gmail MCP server. Configure it in your script via `MultiServerMCPClient` (from `langchain_mcp_adapters`).

What the agent must do:

1. Accept as input: a **recipient email**, a **recipient name**, and a **context string**.
2. Use an LLM node to **draft a personalised cold email** based on the context (subject line + body).
3. Use a **review node** that checks the draft: if it is too generic or too short (your choice of threshold), regenerate. If it passes → send.
4. Call the Gmail MCP tool to **actually send the email**.

The LangGraph graph must have at least: draft → review → (conditional) → send / loop back to draft.

Demonstrate by sending one cold email (to yourself or a test address is fine).

Resources for this assignment are listed on the Notion page. You are encouraged to re-watch or re-read any of them as needed. Reach out on the Whatsapp group or the MS Teams channel for any doubt.

All the best!